

CADERNO DE PROBLEMAS

29/10/2025 – 14h às 17h30

Leia atentamente estas instruções antes de iniciar a resolução dos problemas. Este caderno é composto de 6 problemas, sendo que 2 deles estão descritos em inglês.

1. Não é permitido o uso de material impresso, livros, apostilas e dicionários. Apenas é permitido o uso de lápis, caneta, lapiseira, borracha, régua e papel para rascunho (o papel é fornecido pela comissão organizadora). O acesso à internet é bloqueado durante a realização da prova. A ajuda do ambiente de desenvolvimento como C, C++, Java ou Python pode ser utilizada.

2. A correção das resoluções dos problemas será automatizada, por meio do sistema de submissão eletrônica BOCA, tendo como base os resultados obtidos a partir de uma série de execuções dos algoritmos submetidos pelos times.

3. Siga atentamente as exigências da tarefa quanto ao formato da entrada e da saída do seu algoritmo. Não implemente nenhum recurso gráfico nas suas soluções (janelas, menus, etc.), nem utilize qualquer rotina para limpar a tela ou posicionar o cursor.

4. Os problemas não estão ordenados por ordem de dificuldade neste caderno. A sugestão é de que procure resolver primeiro os problemas mais fáceis.

5. Utilize os nomes indicados nos problemas para nomear os seus arquivos-fonte, de acordo com a linguagem de programação utilizada.

6. Os problemas devem ser resolvidos utilizando o raciocínio entrada-processamento-saída, ou seja, não é necessário armazenar toda a saída para depois exibi-la.

7. Dicas de leitura (entrada de dados) e exibições (saída de dados) encontram-se no verso desta folha.

Faculdade de Informática de Presidente Prudente

http://www.unoeste.br/fipp/maratona

Observações:

Não utilize arquivos para entrada ou saída. Todos os dados devem ser lidos da entrada padrão (teclado) e escritos na saída padrão (tela). Utilize as funções padrão para entrada e saída de dados, como os exemplos a seguir:

- em C: `scanf`, `getchar`, `printf`, `putchar`;
  Para ler um **char** use: `scanf(" %c",&c);` não use: `fflush(stdin); scanf("%c",&c);`

- em C++: as mesmas de C ou os objetos `cin` e `cout`;
  Em C ou C++: use `sprintf(str,"%d",num);` em vez de `itoa(num,str,10);`

  Em C ou C++: não use `include<conio.h>`

- em Java, pode-se usar a classe `Scanner`:

```
Scanner sc = new Scanner(System.in);
int i = sc.nextInt();
String s = sc.next();
float f = sc.nextFloat();
```

  Ou ainda as classes `InputStreamReader` e `BufferedReader`:

```
InputStreamReader isr = new InputStreamReader(System.in);
BufferedReader br = new BufferedReader(isr);
try {
    v = br.readLine();
} catch(Exception e){  }
```

  Para exibição:

```
System.out.println();
System.out.printf("%d", i); //também %f, %c, %s, ...
```

  Em Java: não utilize `package`, o arquivo fonte deve ter a classe com os métodos e o `main`.

- em Python:
  Para leitura de um **int** use: `i = int(input())`
  Para leitura de um **float** use: `f = float(input())`
  Para leitura de um **char** use: `c = input()[0]`
  Para leitura de valores sequencias (quantidade fixa): `10 20 30`

```
v1, v2, v3 = list(map(int, input().split()))
```

  Para leitura de valores sequencias (quantidade não fixa): `10 20 30 ....`

```
valores = list(map(int, input().split()))
for v in valores:
    print(v) # faz alguma coisa com os itens da lista
```

  Para leitura de várias strings (quantidade não fixa): `aaa bbbb ccc`

```
strings = list(map(str, input().split()))
for s in strings:
    print(s) # faz alguma coisa com as strings da lista
```

A Batalha dos Algoritmos (azul)

Problema
A

Nome do arquivo fonte se o código for em Java: *Batalha.java*

Na grande cidade de Datópolis, onde todos os sistemas são movidos a inteligência artificial, dois algoritmos lendários discutem há anos quem é o mais eficiente no processamento de dados: Sir Linear, mestre da Regressão Linear com Gradiente Descendente, famoso por sua disciplina e passos calculados; Lord Vizinho, o poderoso K-Nearest Neighbors (KNN), conhecido por comparar tudo e todos antes de tomar qualquer decisão.

Cansados de discutir por séculos, eles decidiram resolver isso da forma mais científica possível. Eles receberão a mesma base de dados com N linhas e M colunas — e quem tiver o menor custo computacional será declarado o campeão supremo de Datópolis!

Os custos são definidos pelo Conselho de Complexidade Computacional:

Sir Linear executa seu treinamento com custo:

$Custo_A = N * M * 100$

Lord Vizinho precisa comparar cada exemplo com todos os outros durante a predição, resultando em:

$Custo_B = N * N * M$

Sua missão, jovem programador, é determinar qual deles vence esse duelo!

Entrada

Dois inteiros N e M ($1 \leq N, M \leq 10^3$) representando respectivamente o número de linhas e colunas da base de dados.

Saída

Uma única palavra:

"A" — se Sir Linear for mais eficiente

"B" — se Lord Vizinho for mais eficiente

"IGUAL" — se houver empate técnico

Exemplo de entrada	**Saída para o exemplo de entrada**
`1000 10`	`A`

Exemplo de entrada	**Saída para o exemplo de entrada**
`10 10`	`B`

O Mercado dos Parâmetros (branca)

Problema
B

Nome do arquivo fonte se o código for em Java: *Mercado.java*

Em Datópolis, existe um Mercado onde comerciantes vendem pares de parâmetros (a, b) para modelos de regressão linear. Cada comprador traz sua base de dados com N observações (x_i, y_i).

Sua missão é avaliar vários pares (a, b) candidatos e dizer qual deles gera o menor erro médio ao prever $\hat{y}$ com a fórmula:

$$\hat{y} = a \cdot x + b$$

O erro é medido usando Erro Quadrático Médio (MSE):

$$\text{MSE}(a, b) = (1/N) * \Sigma (y_i - \hat{y}(x_i))^2$$

Entrada

A entrada é composta por:

Uma linha contendo dois inteiros N e K, representando respectivamente a quantidade de observações e a quantidade de pares candidatos.

As próximas N linhas contêm dois números reais x e y, representando os pontos da base de dados. As próximas K linhas contêm dois números reais a e b, representando os parâmetros de cada modelo.

Saída

Você deve imprimir apenas um número inteiro, que representa o índice (começando em 1) do par (a, b) com o menor MSE. Em caso de empate, imprima o índice do primeiro que atingiu o menor erro.

Exemplo de entrada	**Saída para o exemplo de entrada**
2 3 0 1 2 3 0 0 1 1 0.5 0.75	2

Nota

Par (a, b) ($\hat{y}(x) = a*x+b$)	**Previsões $\hat{y}(0)$**	**Previsões $\hat{y}(2)$**	**Erros ao quadrado**	**MSE**
(0, 0) = 0*x + 0	0 = 0*0 + 0	0 = 0*2 + 0	$(1-0)^2 + (3-0)^2 = 10$	10 / 2 = 5.0
(1, 1) = 1*x + 1	1 = 1*0+1	3 = 1*2 + 1	$(1-1)^2 + (3-3)^2 = 0$	0 / 2 = 0.0
(0.5, 0.75) = 0.5*x + 0.75	0.75 = 0.5*0 + 0.75	1.75 = 0.5*2 + 0.75	$(1-0.75)^2 + (3-1.75)^2 = 1.625$	1.625 / 2 ≈ 0.8125

The Space Explorers (amarela)

Problema C

Source file name if the code is in Java: *Space.java*

The Galactic Federation is organizing planets into regions based on their coordinates in the stellar plane (*x*, *y*). To do this, the K-Means algorithm was used, which grouped all planets into *K* clusters.

Each cluster has a centroid (the midpoint of the group). Now, several explorers are arriving with new coordinates and need to know which region (cluster) they belong to. Your mission is to answer *Q* queries, informing which cluster is closest to each point provided. Using Euclidean distance to measure distance:

$$d(Euc) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Input

A line with an integer *K* — number of clusters ($1 \leq K \leq 1000$). The next *K* lines, each with two real numbers *cx*, *cy* — coordinates of the cluster centroids ($-10^4 \leq cx, cy \leq 10^4$).

A line with an integer *Q* — number of queries ($1 \leq Q \leq 10^4$). The following *Q* lines, each with two real numbers *x*, *y* — coordinates of the query points ($-10^4 \leq x, y \leq 10^4$).

Output

For each query, print a line with a single integer: the index (1-based) of the closest cluster. In case of a tie, select the cluster with the lowest index.

Input example	Output for the input example
3 0 0 5 5 -3 4 2 1 1 6 5	1 2

Note

Point	Distance to Cluster 1	Distance to Cluster 2	Distance to Cluster 3	Result
(1,1)	$\sqrt{2} \approx 1.41$	$\sqrt{32} \approx 5.66$	$\sqrt{25} \approx 5$	1
(6,5)	$\sqrt{61} \approx 7.81$	$\sqrt{1} = 1.00$	$\sqrt{82} \approx 9.05$	2

The Supreme AI Model (verde)

Problema
D

Source file name if the code is in Java: *Model.java*

You are a machine learning engineer working in an AI research lab. Your goal is to train a generative number model, which creates integers from input sequences.

However, the model can only generate numbers by putting other numbers together in sequence. To achieve the highest validation score, you need to find out what is the largest possible number that the model can form from a list of integers generated by different modules of the system.

For example, given the list [3, 3, 4, 5, 9], the model can combine the numbers into "95433" to maximize the output.

Your mission is to program the AI lab to automatically organize the numbers to form the largest possible number.

Input

The input consists of:

A line with an integer N — the amount of numbers provided by the system modules ($1 \leq N \leq 10^5$).

A line with N integers a_1, a_2, ..., a_N ($0 \leq a_i \leq 9$) — he numbers generated by the modules.

Output

Print a single line containing the largest possible integer formed by concatenating the numbers.

Input example	Output for the input example
4 1 0 2 0	2100

Estação Mais Próxima (rosa)

Problema
E

Nome do arquivo fonte se o código for em Java: *Estacao.java*

Você trabalha como analista de dados em uma empresa ferroviária que administra uma linha de trem que segue uma única direção — de norte a sul — como se fosse uma linha reta, onde cada estação tem uma posição única representada por um número inteiro ao longo dessa linha (imagine um único eixo *Y*).

Com um novo sistema de monitoramento sendo implementado, sensores móveis estão sendo testados ao longo da ferrovia. A cada momento, esses sensores reportam sua posição atual, e o sistema precisa identificar rapidamente qual é a estação mais próxima para tomada de decisões (como alertas de emergência, manutenção ou atendimento).

A empresa exige que o tempo de resposta seja mínimo, então você deve implementar um algoritmo eficiente para calcular, para cada posição de sensor recebida, a menor distância possível até uma estação já existente na malha ferroviária. A distância entre dois pontos ao longo dessa ferrovia é dada pela distância Manhattan em 1D, ou seja, distância(a, b) = $|a - b|$.

Entrada

A primeira linha contém dois inteiros N e Q ($1 \leq N, Q \leq 10^5$):

A segunda linha contém N inteiros **ordenados em ordem crescente** $y_1, y_2, ..., y_N$ ($-10^9 \leq y_i \leq 10^9$): as posições das estações fixas ao longo da ferrovia.

As próximas Q linhas contêm, cada uma, um inteiro q_j ($-10^8 \leq q_j \leq 10^8$): a posição do sensor no momento da consulta.

Saída

Para cada consulta, imprima uma linha com um único inteiro: A menor distância entre o sensor e qualquer estação existente.

Exemplo de entrada	**Saída para o exemplo de entrada**
3 2 -100 0 1500 2000 -200	500 100

A Curiosidade de Lee **(vermelha)**

Problema
F

Nome do arquivo fonte se o código for em Java: *Lee.java*

Lee é um estudante de ciência de dados que vem estudando vetores em espaços de múltiplas dimensões — como os usados para representar dados em algoritmos de machine learning.

Durante uma tarde estudando vetores, ele ficou curioso com uma pergunta simples:

"Será que, entre um conjunto de pontos em até 10 dimensões, existe algum par cujo ponto médio também seja um ponto com todas as coordenadas inteiras?"

Para matar a curiosidade, Lee resolveu escrever um programa que recebe uma lista de vetores com coordenadas inteiras e determina se existe pelo menos um par de pontos cujo ponto médio também tem todas as coordenadas inteiras.

Agora ele quer que você o ajude com esse programa.

Definição de ponto médio

Dado dois pontos $A=(a_1, a_2,..., a_d)$ e $B=(b_1, b_2,..., b_d)$, o ponto médio entre eles é:

$$M = ((a_1 + b_1)/2, (a_2 + b_2)/2,..., (a_d + b_d)/2)$$

Entrada

Um inteiro N — número de pontos ($2 \leq N \leq 10^5$)
Um inteiro D — número de dimensões ($1 \leq D \leq 10$)
Em seguida, N linhas, cada uma com D inteiros: as coordenadas de um ponto. Cada coordenada está no intervalo $0 \leq x \leq 10^9$.

Saída

"SIM" se existe ao menos um par de pontos com ponto médio inteiro.
"NAO" caso contrário.

Exemplo de entrada	**Saída para o exemplo de entrada**
3 2 1 2 3 4 2 2	SIM

Explicação: Os pontos (1, 2) e (3, 4): ((1 + 3)/2, (2 + 4)/2) = (4/2, 6/2) = (2, 3) que são coordenadas inteiras.